

JC4003 Natural Language Processing

Lecture 13: Automatic Speech Recognition and Text-to-Speech 1

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Outline

Speech Representation

Acoustic characteristics of voice (review)

Automatic Speech Recognition (ASR)

Text-to-Speech (TTS)

Speech Representation

Speech Representation

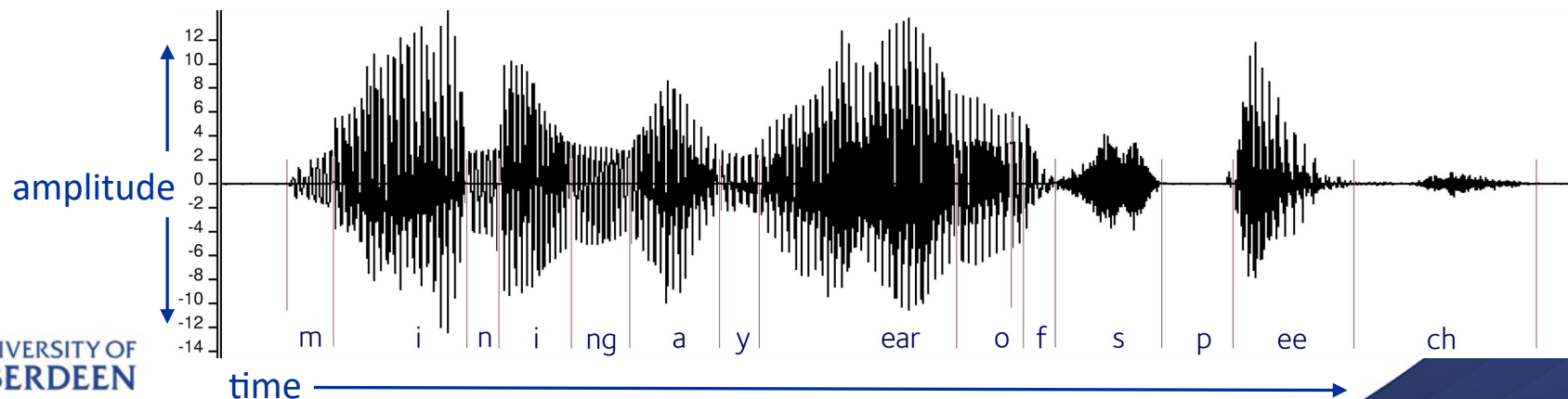
Speech representation refers to **converting a speech signal into a mathematical or data form** that can be processed, analysed, or used for model training.

Effective speech representations retain essential information from the speech while eliminating unnecessary data, making it suitable for tasks like Automatic Speech Recognition (ASR), Text-to-Speech (TTS), and other speech processing applications.

Waveform Representation

Waveform representation is **the most basic and direct way** of representing a speech signal. It involves capturing the **raw sound wave** as it propagates through the air, typically by measuring the amplitude of the sound wave over time.

This representation reflects the physical properties of the sound, such as changes in air pressure caused by the vibration of the speaker's vocal cords.



Waveform Representation (cont.)

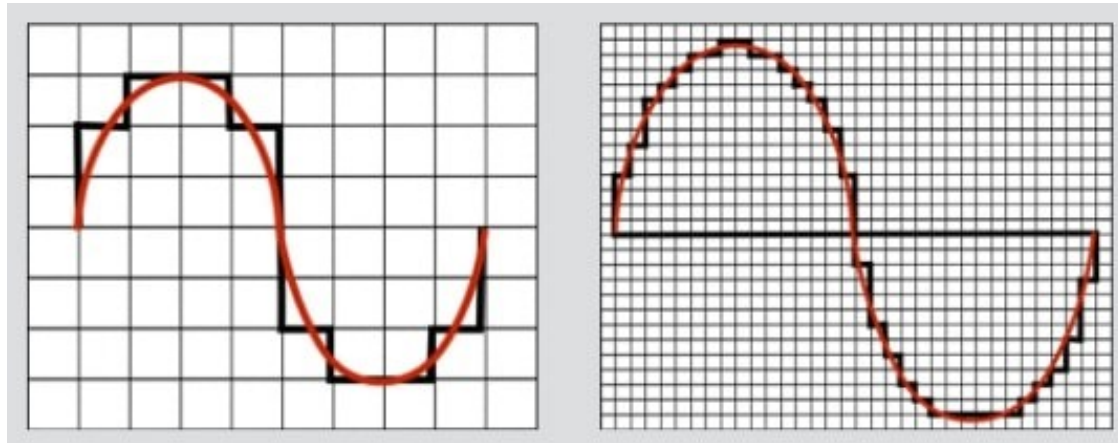
Key Characteristics of Waveform Representation:

- **Time-Domain Representation:** A waveform is a plot of sound amplitude as a function of time. It shows how the loudness of the sound varies over time, capturing the raw acoustic signal that can be recorded through a microphone.
- **Amplitude:** The amplitude of the waveform represents the energy or loudness of the sound at each sample point. It is usually stored as a numerical value, with higher values indicating louder sounds.
- **Sampling Rate:** The waveform is typically represented as a series of discrete samples taken at regular intervals. The sampling rate (measured in Hertz) determines how many samples are captured per second. A higher sampling rate allows for a more accurate representation of the original sound but requires more storage and processing power.

Sampling Rate

The sampling rate, also known as the sample rate or sampling frequency, is defined as the **number of samples taken per second from a continuous signal to make it discrete**.

It is measured in **Hertz (Hz)**, where 1 Hz represents one sample per second. For instance, a sampling rate of 16 kHz means that 16,000 samples of the signal are taken every second.



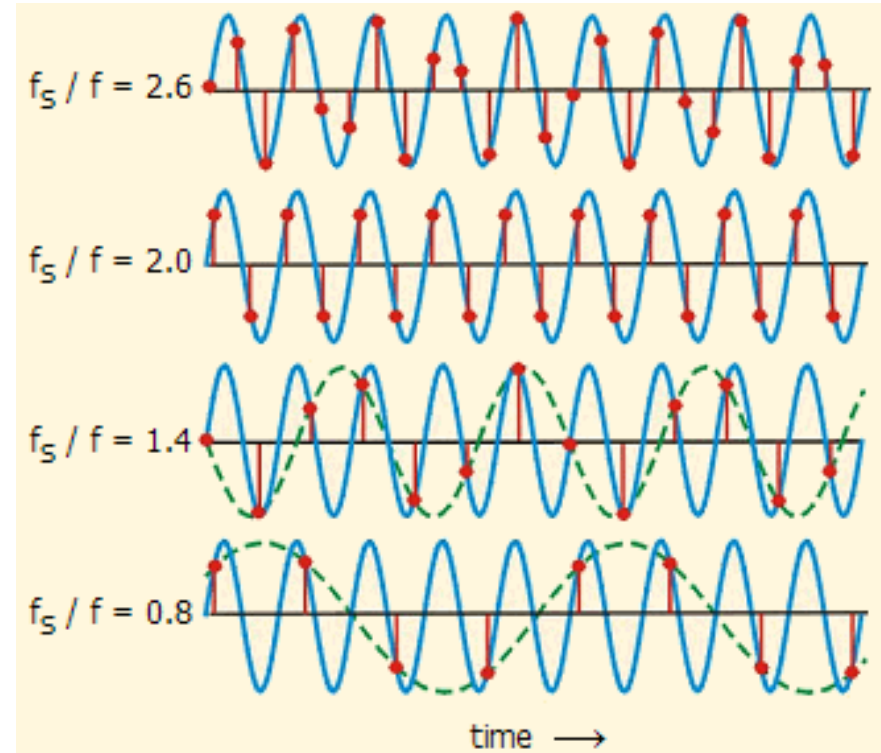
Low sampling rate

High sampling rate

Sampling Rate (cont.)

According to the [Nyquist-Shannon sampling theorem](#), to accurately capture all the information in an analog signal, the [sampling rate must be at least twice the highest frequency present in the signal](#).

This minimum rate is called the [Nyquist rate](#). For example, human speech typically contains frequencies up to about 8,000 Hz, so a sampling rate of at least [16,000 Hz](#) (16 kHz) is necessary to capture it accurately.



Sampling of a sinusoidal signal of frequency f at different sampling rates f_s

Waveform Representation (cont.)

Advantages:

- **Richness of Information:** Waveform representation captures **all the details of the original sound signal**, including all frequencies, amplitudes, and phase information.

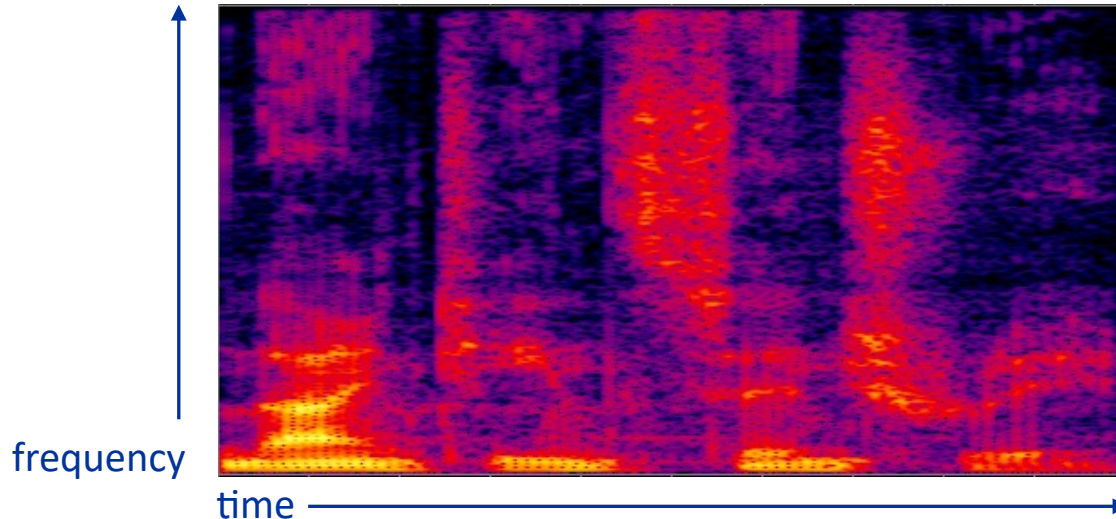
Limitations:

- **High Data Volume:** Because waveforms contain raw, unprocessed data, they **require significant storage space**, especially at high sampling rates and bit depths.
- **Not Suitable for Direct Processing:** Raw waveforms contain too much **redundant information** for many speech processing tasks.

Short-Time Fourier Transform (STFT)

The Short-Time Fourier Transform converts the signal from the time domain to the frequency domain but in a way that retains time information as well.

STFT provides a time-frequency representation of the signal, allowing us to see how the frequency content of the signal evolves over time.



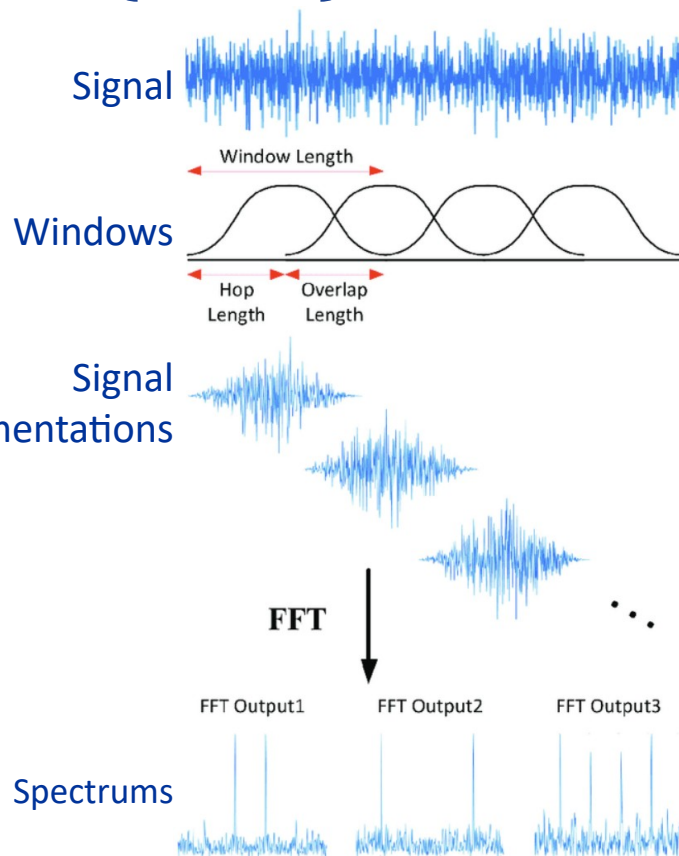
Short-Time Fourier Transform (cont.)

STFT breaks down a signal into smaller segments (or windows) and then applies the Fourier Transform to each segment.

Windowing: The signal is divided into overlapping short segments or windows.

Fourier Transform: For each windowed segment, the Fourier Transform converts the time-domain signal into a frequency-domain representation. The results are spectrums showing the frequency components present within that window.

Time-Frequency Matrix: how the signal's frequency content changes over time.



Short-Time Fourier Transform (cont.)

Advantages:

- **Time-Frequency Analysis:** STFT allows us to observe both time and frequency characteristics of a signal simultaneously.
- **Localised Frequency Information:** STFT provides localised frequency information, which is useful when the signal's frequency content changes rapidly over time.

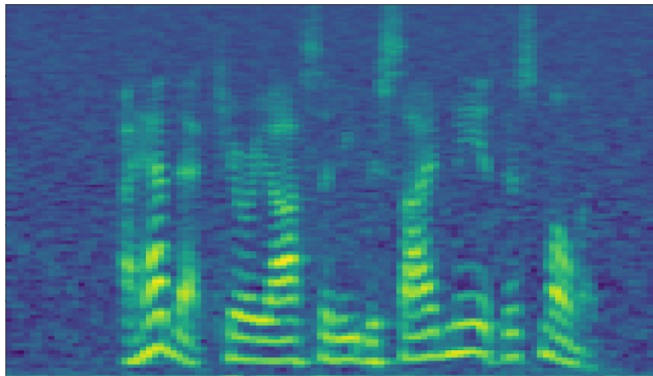
Disadvantages:

- **Time-Frequency Trade-off:** The resolution of the time-frequency analysis depends on the window size. A larger window provides better frequency resolution but poorer time resolution, while a smaller window improves time resolution at the cost of frequency resolution. This trade-off is a fundamental limitation of STFT.

Log-Mel Spectrogram

A Log-Mel-spectrogram (also Mel-spectrogram) is created by applying a [Short-Time Fourier Transform \(STFT\)](#) to a speech signal, generating a spectrogram, and [then mapping the frequency axis to the Mel scale](#), and a logarithmic transformation is often applied to the Mel-spectrogram as Log-Mel-spectrogram.

The Mel scale is designed based on [human auditory perception](#), with higher resolution at lower frequencies and lower resolution at higher frequencies, [as the human ear](#) is more sensitive to low frequencies than high frequencies.



The Mel Spectrogram and STFT graph look similar.

Log-Mel Spectrogram (cont.)

Advantages:

- **Aligned with Human Perception:** The Mel scale is more in tune with human auditory perception. This makes it ideal for tasks where human listening is involved.
- **Compact Representation:** The Mel-spectrogram compresses the frequency information, making it more compact than traditional linear spectrograms.

Disadvantages:

- **Loss of Fine Frequency Detail:** For tasks that require precise frequency analysis (e.g., certain music applications or high-resolution audio processing), this loss of detail can be a limitation.

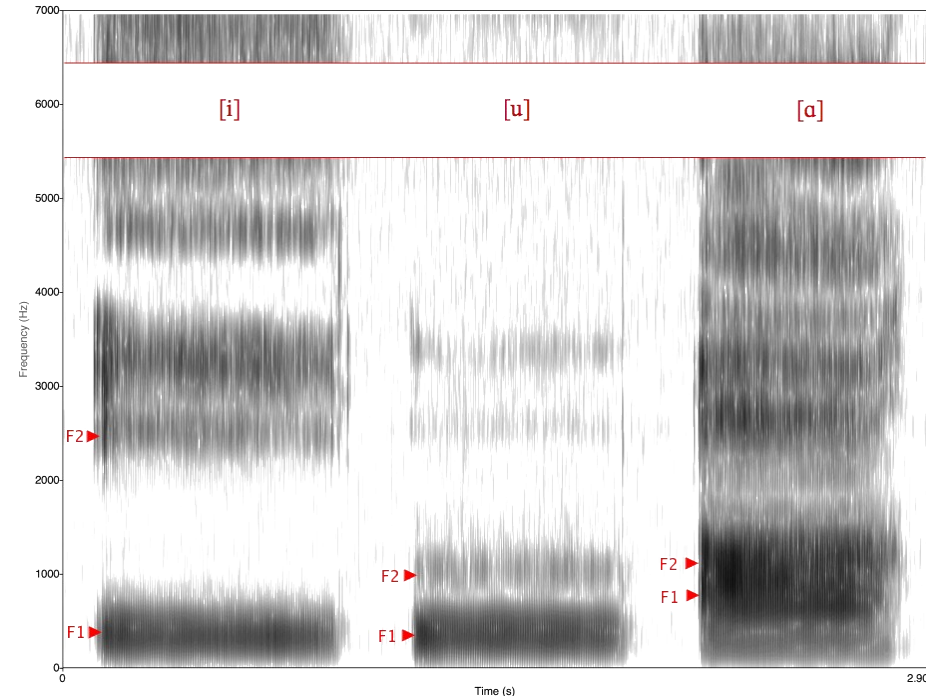
Acoustic Characteristics of Voice (review)

Acoustic Characteristics of Vowels

Different vowels are characterised by the relative values of **formants**, which are the acoustic resonances of the vocal tract and appear as dark bands on a spectrogram.

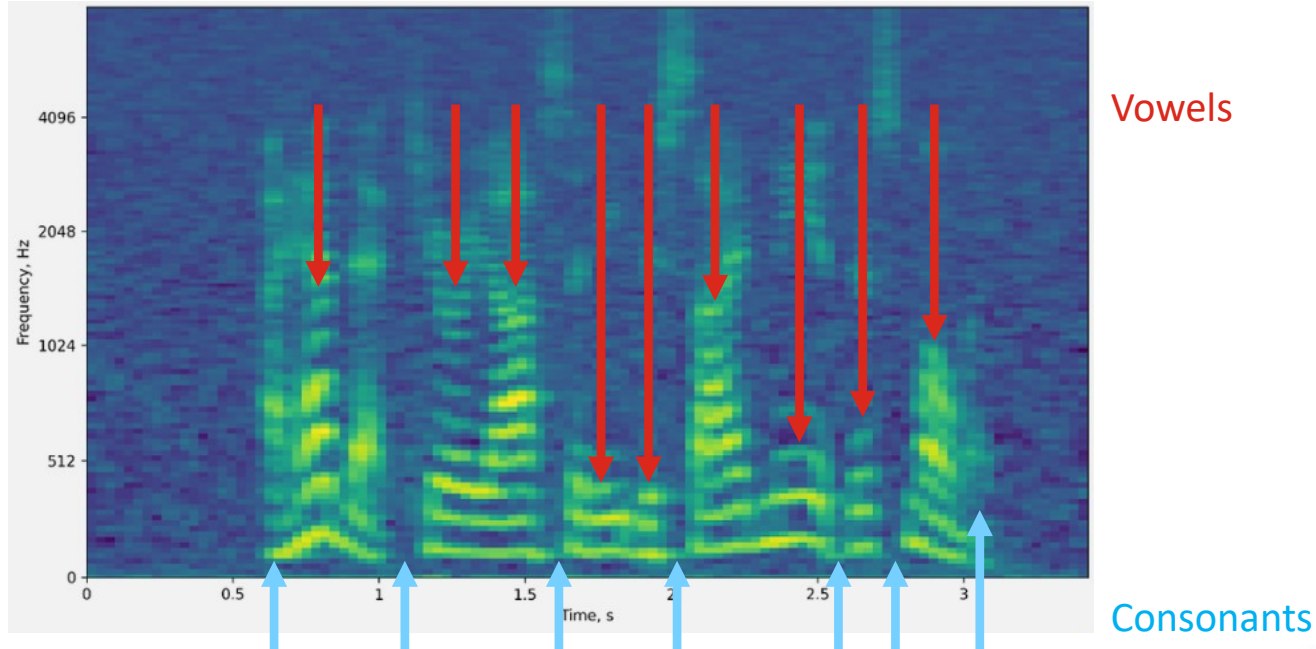
The vocal tract acts as a resonating chamber, and the position of the jaw, lips, and tongue alters its dimensions, thereby affecting the formant frequencies produced.

The acoustic properties of vowels can be visualised using a **spectrogram**, which displays the energy present at each frequency and its variation over time.



Acoustic Characteristics of Consonants

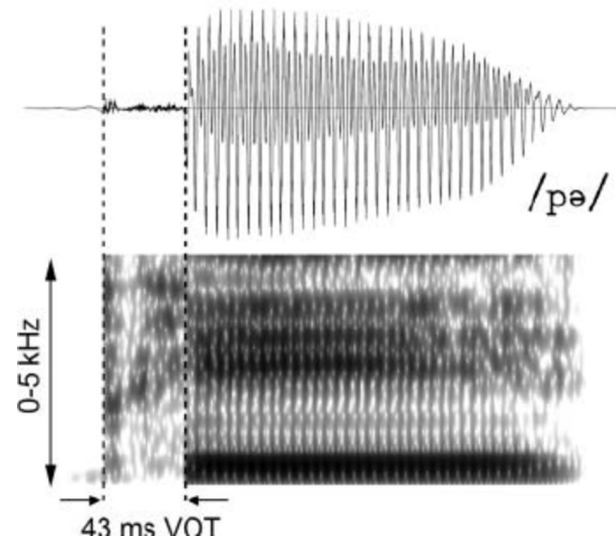
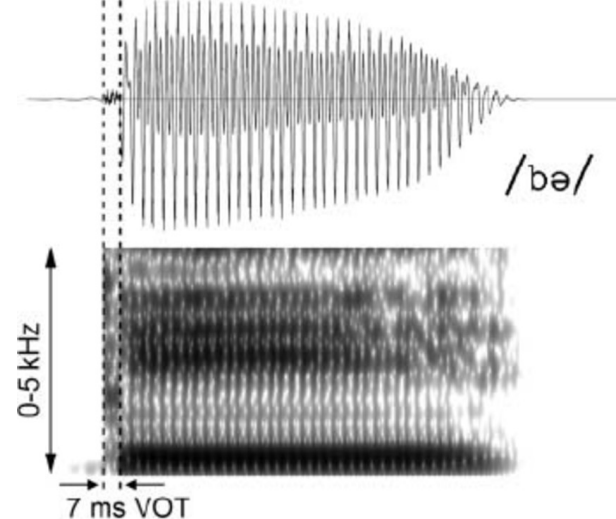
Consonants **do not** have obvious formant structures in the mel-spectrogram.



Voice Onset Time (VOT)

Voice Onset Time (VOT) refers to the length of time that passes between the release of a consonant and the onset of vocal cord vibration, which is crucial in distinguishing between voiceless and voiced consonants in many languages.

- **Pre-voicing:** The vowel starts before the consonant is released, resulting in a negative VOT value (e.g. -10ms).
- **Short-lag:** The vowel starts shortly after the release of the consonant (e.g. VOT = 10ms).
- **Long-lag:** Here, there is a longer delay before the vowel after the release of the consonant (e.g. VOT = 50ms).

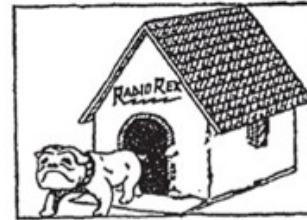


Automatic Speech Recognition (ASR)

The First Speech-recognition Machines

Understanding spoken language has been a fundamental aim of computer language processing from the start. In fact, **speech processing began long before computers existed**.

One of the first speech-recognition machines was a toy from the **1920s** called "**Radio Rex**". It was a small celluloid dog that moved when it **detected sound waves at around 500 Hz**, which matched the vowel sound [eh] in "Rex." So, it appeared as if the dog responded when called.



"Radio Rex"
\$1.98

—Put' him in his kennel,
then shout "Rex," and he
immediately jumps out at
you! But he won't bite.



Automatic Speech Recognition

Automatic Speech Recognition (ASR) refers to the technology that enables computers and other devices to **recognise and transcribe spoken language into text**.

ASR plays a crucial role in various applications, such as **virtual assistants** (e.g. **Siri**, **Alexa**, or **Google Assistant**), **transcription services**, and **automated customer service systems**.

Modern ASR systems are designed to handle various accents, which can be quite diverse. However, achieving high accuracy in transcription, especially with strong **accents** or in **noisy** environments, remains a **challenge**.



Automatic Speech Recognition

Automatic Speech Recognition (ASR) systems can be classified into several types based on the underlying model architecture and approach they use:

- **Template-based ASR (1960s-1970s)**: This is one of the earliest forms of ASR. Each word or phrase has a template, and the system compares the incoming speech to these templates to find the best match. E.g. Radio Rex
- **Statistical ASR i.e. HMM-based ASR (1980s-2000s)**: In Hidden Markov Models (HMM), speech is modelled as a sequence of states (representing different sounds or phonemes). They are relatively robust but can struggle with complex language patterns.
- **Neural Network-based ASR (2000s-Present)**: These systems use deep neural networks to model the relationship between audio features and the corresponding text.

Neural Network-based ASR

Neural Network-Based ASR functions **similarly to other sequence-to-sequence tasks**, such as machine translation, where many neural network models can be used (e.g., RNN, LSTM, or Transformer).

The key distinction with ASR is that the **input sequences are typically very long** (representing continuous speech), while the **output sequences are relatively short** (which are the transcriptions).

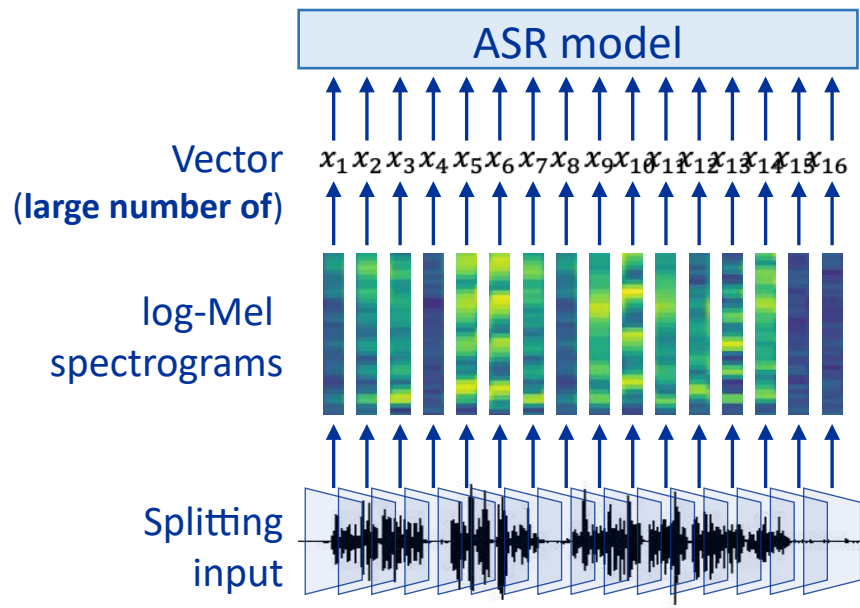
This **imbalance between** input and output length presents unique challenges in modelling.

The Inputs of ASR Models

The inputs of ASR models are derived from audio signals, which are transformed into numerical vectors through a series of processing steps.

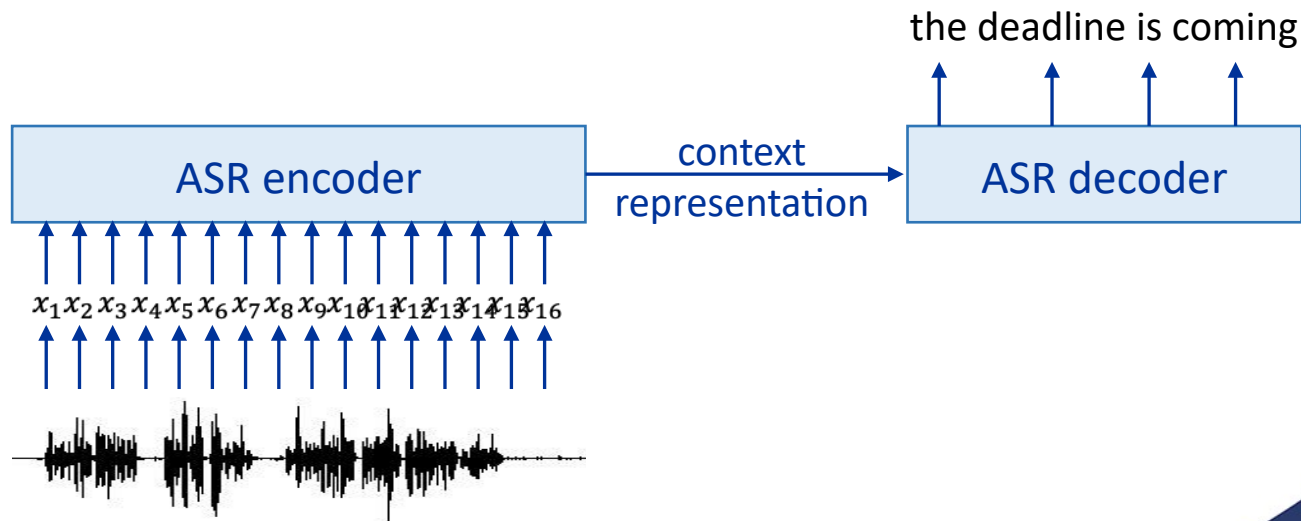
Initially, the raw waveform of the audio is converted into a log-Mel spectrogram, then further processed into feature vectors that serve as the input for the ASR model.

To ensure accuracy, the audio waveform is split into small time segments, typically around 25 milliseconds each, with overlap. This results in a large number of segments, making the input sequence much longer.



Encoder-decoder Models for ASR

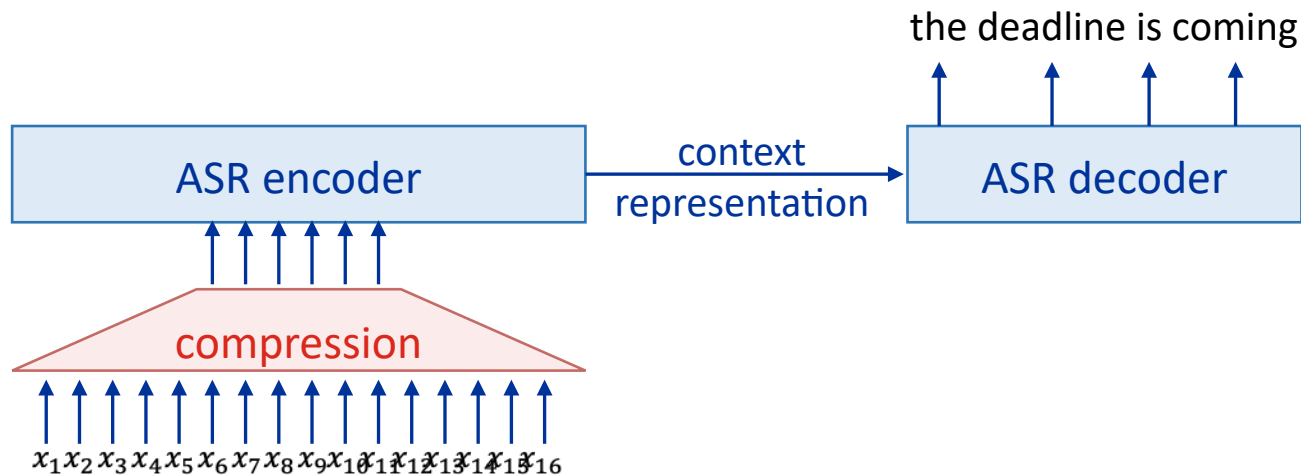
The **basic architecture for ASR** is the encoder-decoder (implemented with either RNNs or Transformers)



Encoder-decoder Models for ASR (cont.)

Because the length difference of inputs and outputs is so extreme, encoder-decoder architectures for speech need to have a **special compression stage** that shortens the input acoustic feature sequence before the encoder stage.

Otherwise, model training will be difficult.



Connectionist Temporal Classification (CTC)

CTC-based Models for ASR

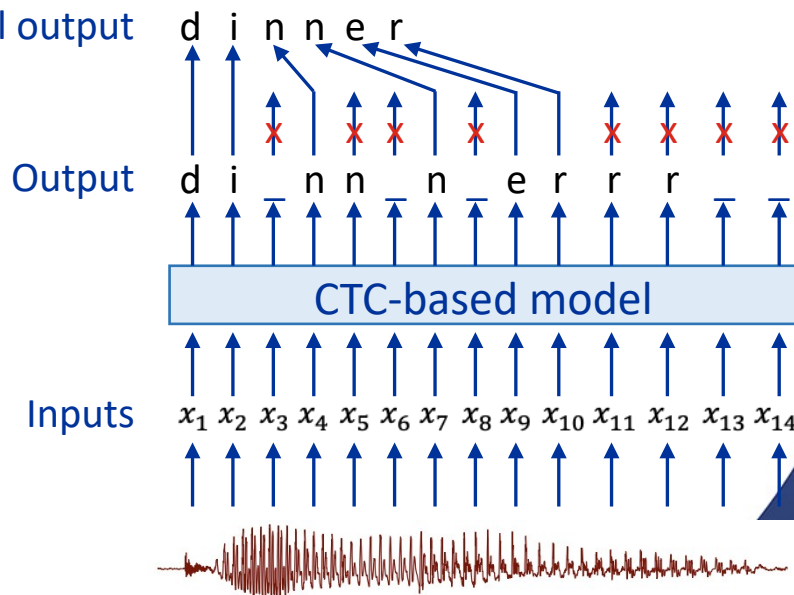
CTC-based models are tailored for ASR, handling significant variation between input and output lengths.

CTC-based models treat ASR as a **sequence labelling task**, assigning a phonetic symbol to each audio segment.

CTC introduces a **blank symbol (_)**. The models are trained to use blanks to **annotate non-transcription segments** and **separate letters**.

In the final output, **blanks are ignored**, and **consecutive duplicate letters are merged**, yielding the correct transcription.

Final output



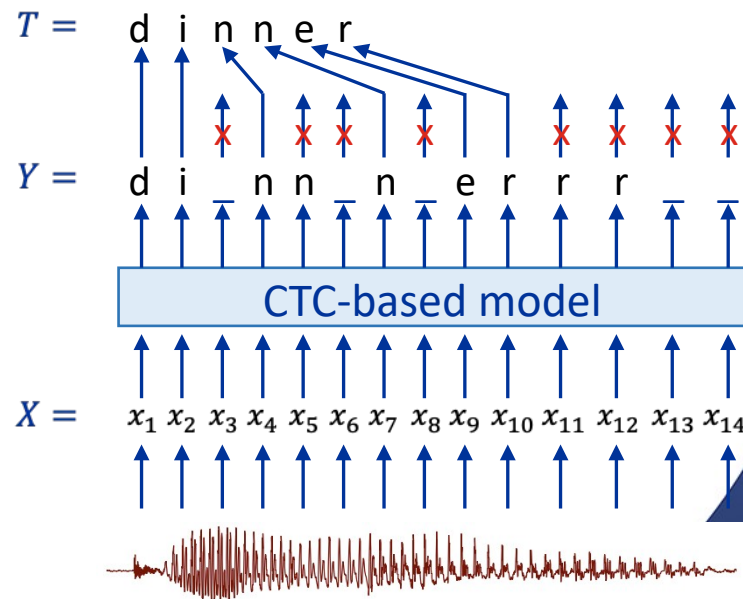
Training -- CTCLoss

CTC-based models require the use of **CTCLoss** during training.

CTCLoss takes as input the **predicted probabilities** (Y) from the model for each possible letter (including the blank symbol) at each time step and the **target transcription sequence** (T).

$$\mathcal{L} = CTCLoss(Y, T)$$

where Y is a sequence of vector representation the letter distribution, and T is the ground truth transcription sequence.



CTCLoss

The model's output Y can have **multiple possible** sequences that map to the same target T because different alignments, including blanks and repeated characters, can produce the same final transcription after blanks are removed and identical letters are merged, e.g.:

Y	T	Y	T
di_nn_n_errr__	dinner	di_nn_n_errr__	dinner
diinnnn_ner__	dinner	diinnnn_ner__	dinner
_din_n_er__	dinner	_din_n_er__	dinner
di_nn_n_errr__	✓	dinner	
diinnnn_ner__	✓	dinner	
_din_n_er__	✓	dinner	

These possible outputs are all valid.

CTCLoss (cont.)

CTCLoss works by **maximising the combined probability of all** the valid outputs. The probability of each valid outputs in CTC is calculated by multiplying the probabilities of the individual symbols (including blanks) at each time step along that sequence.

Y	$P(Y)$	Y	$P(Y)$
di_nn_n_errr__	$P(Y_1) = P(d)P(i d)P(_ di)P(n di) \dots$	di_nn_n_errr__	$P(Y_1) = P(d)P(i d)P(_ di)P(n di) \dots$
diinnnn_ner__	$P(Y_2) = P(d)P(i d)P(i i)P(n dii) \dots$	diinnnn_ner__	$P(Y_2) = P(d)P(i d)P(i i)P(n dii) \dots$
_din_n_er____	$P(Y_3) = P(_)P(d _)P(i _d)P(n _di) \dots$	_din_n_er____	$P(Y_3) = P(_)P(d _)P(i _d)P(n _di) \dots$
di_nn_n_errr__	Y	di_nn_n_errr__	$P(Y)$
diinnnn_ner__	di_nn_n_errr__	diinnnn_ner__	$P(Y_1) = P(d)P(i d)P(_ di)P(n di) \dots$
_din_n_er____	diinnnn_ner__	_din_n_er____	$P(Y_2) = P(d)P(i d)P(i i)P(n dii) \dots$
	_din_n_er____		$P(Y_3) = P(_)P(d _)P(i _d)P(n _di) \dots$
di_nn_n_errr__	Y	di_nn_n_errr__	$P(Y)$
diinnnn_ner__	di_nn_n_errr__	diinnnn_ner__	$P(Y_1) = P(d)P(i d)P(_ di)P(n di) \dots$
_din_n_er____	diinnnn_ner__	_din_n_er____	$P(Y_2) = P(d)P(i d)P(i i)P(n dii) \dots$
	_din_n_er____		$P(Y_3) = P(_)P(d _)P(i _d)P(n _di) \dots$
di_nn_n_errr__	Y	di_nn_n_errr__	$P(Y)$
diinnnn_ner__	di_nn_n_errr__	diinnnn_ner__	$P(Y_1) = P(d)P(i d)P(_ di)P(n di) \dots$
_din_n_er____	diinnnn_ner__	_din_n_er____	$P(Y_2) = P(d)P(i d)P(i i)P(n dii) \dots$
	_din_n_er____		$P(Y_3) = P(_)P(d _)P(i _d)P(n _di) \dots$

CTCLoss is defined as the negative logarithm of the sum of the probabilities of all possible alignment paths that lead to the correct transcription.

$$CTCLoss(Y, T) = -\log \sum_i P(Y_i)$$

CTCLoss (cont.)

Because the output sequence is very long, calculating the probabilities of all possible outputs is extremely complex.

Therefore, CTCLoss typically uses a [dynamic programming algorithm](#) to bypass the need to calculate each possible output probability individually. Instead, it directly computes their total sum [efficiently](#).

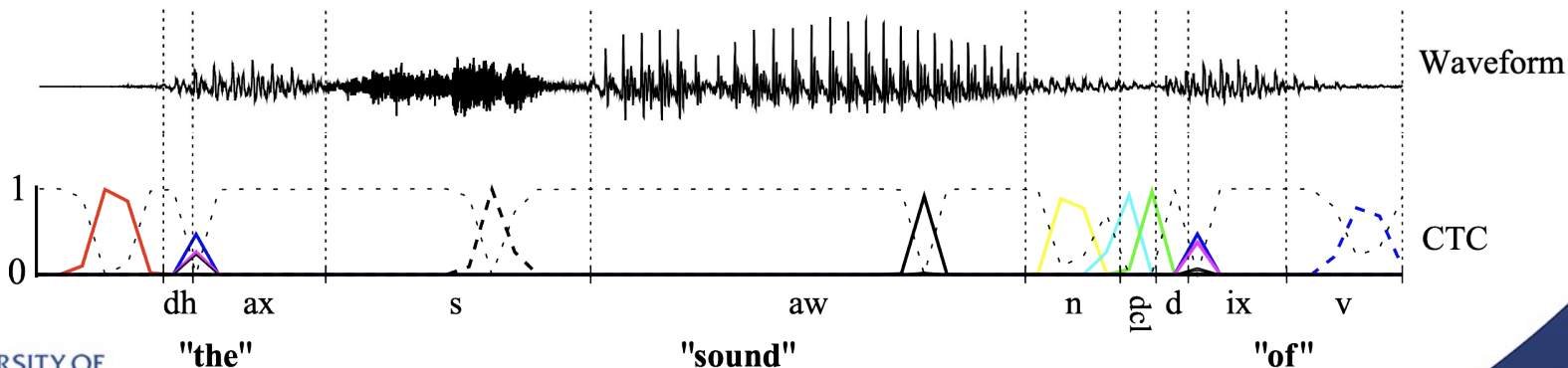
For detailed calculation methods, refer to [Chapter 16 of the textbook](#) or the original paper:

Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006, June). Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning* (pp. 369-376).

CTCLoss (cont.)

Although CTC maximises the probability of all possible outputs, after training with a large number of samples, **models tend to converge towards assigning a single label to a syllable that spans multiple segments, marking its other segments as blanks.**

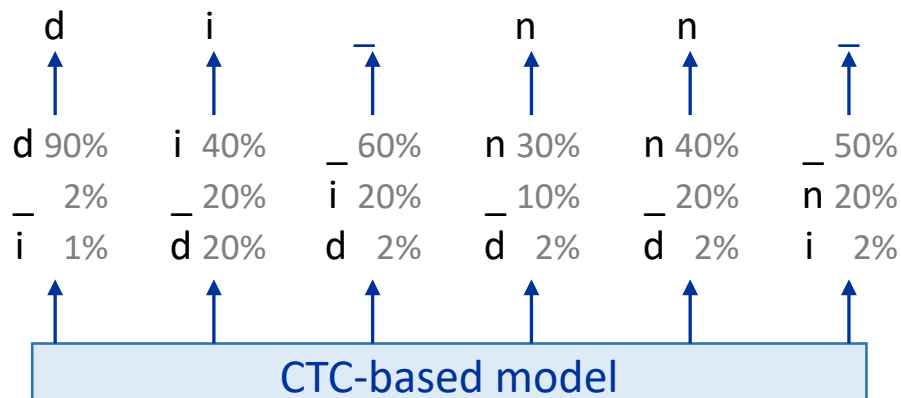
This behaviour allows the model to efficiently represent the transcription by focusing on the most likely label position, while using blanks to handle the temporal variability in the input.



Inference

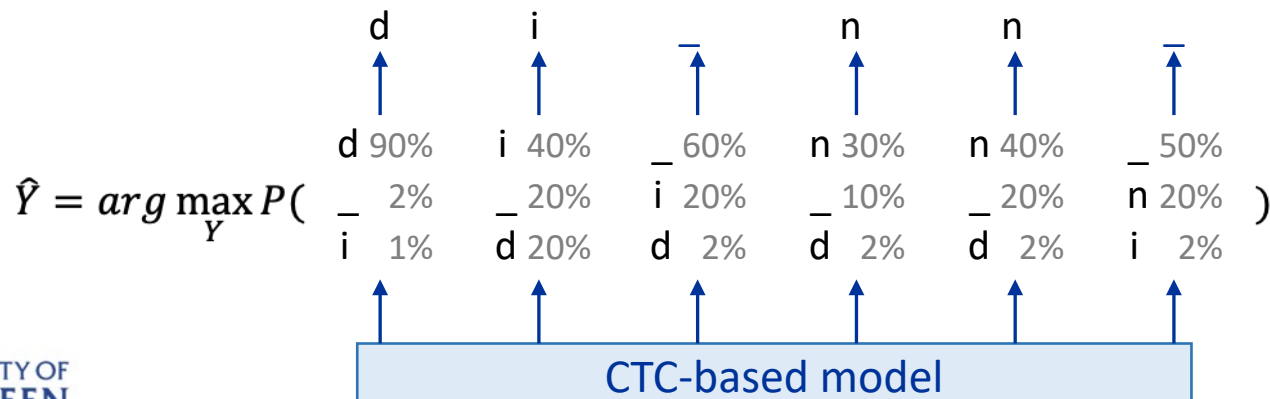
CTC inference refers to the process of **generating the final transcription** from the outputs of a trained model (prediction phase). During inference, the model outputs a sequence of **probabilities for each possible letters** (including the blank symbol) at each time step.

Simply, the most likely transcription is derived by **selecting the highest probability letter** at each step, followed by collapsing consecutive identical characters into one and removing all blank symbols.



Inference (cont.)

To achieve **better results** during CTC inference, instead of always selecting the highest probability at each step, **a probabilistic model or language model** can be introduced to allow the selection of the transcription that **maximises the overall sentence probability**, taking into account the context and likelihood of word sequences, leading to more accurate and natural transcriptions.



Conclusion

Speech Representation: Discussed various ways to represent speech, such as waveforms, STFT, and log-Mel spectrograms, highlighting their roles in ASR and TTS.

ASR Overview: Explored the progression from template-based and statistical models to modern neural network-based ASR systems.

Neural Network-Based ASR: Highlighted the encoder-decoder and CTC-based models, which effectively map speech to text by handling input-output length discrepancies.

CTCLoss and Inference: Introduced CTC Loss as a method for training ASR models and explored inference techniques for producing accurate transcriptions.

Challenges and Applications: Acknowledged the complexities of ASR in real-world environments and its wide applications in virtual assistants, transcription, and customer service.